

Operating System Principles

Module – 2

Lecture 3 - Process

Process

- A process is a program in execution
- It is the foundation for all the computations that happen on your computer
- A process is unit of work that OS manages.
- The status of the current activity of a process is represented by the value of the **program counter** and the contents of the **processor's registers**.

Process

- A program is a ***passive*** entity, such as a file containing a list of instructions stored on disk
- A process is an ***active*** entity, with a program counter specifying the next instruction to execute and a set of associated resources.
- A program becomes a process when an executable file is loaded into memory.

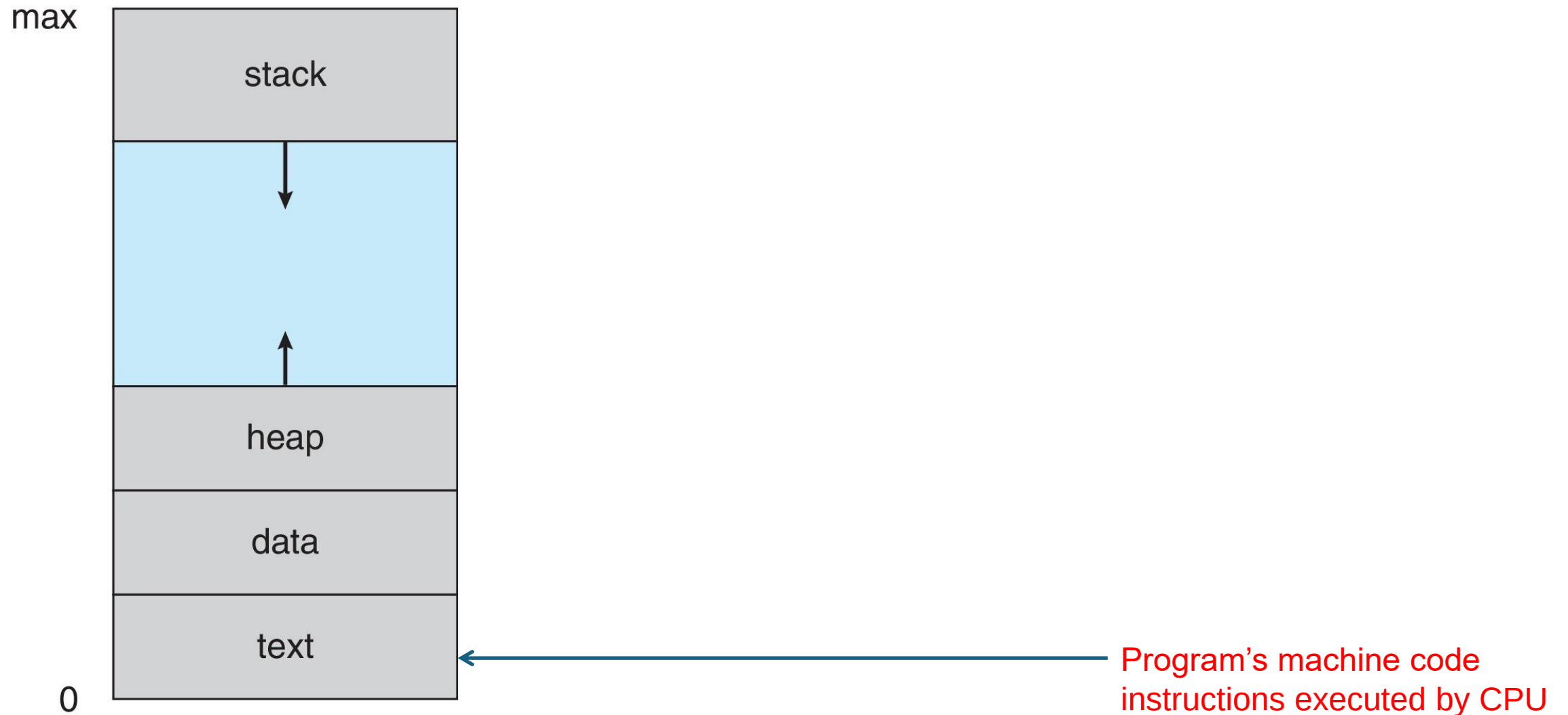
Process

- **Two common techniques** for loading executable files are double-clicking an icon representing the executable file and entering the name of the executable file on the command line.
- Although two processes may be associated with the same program, they are nevertheless considered two separate execution sequences.

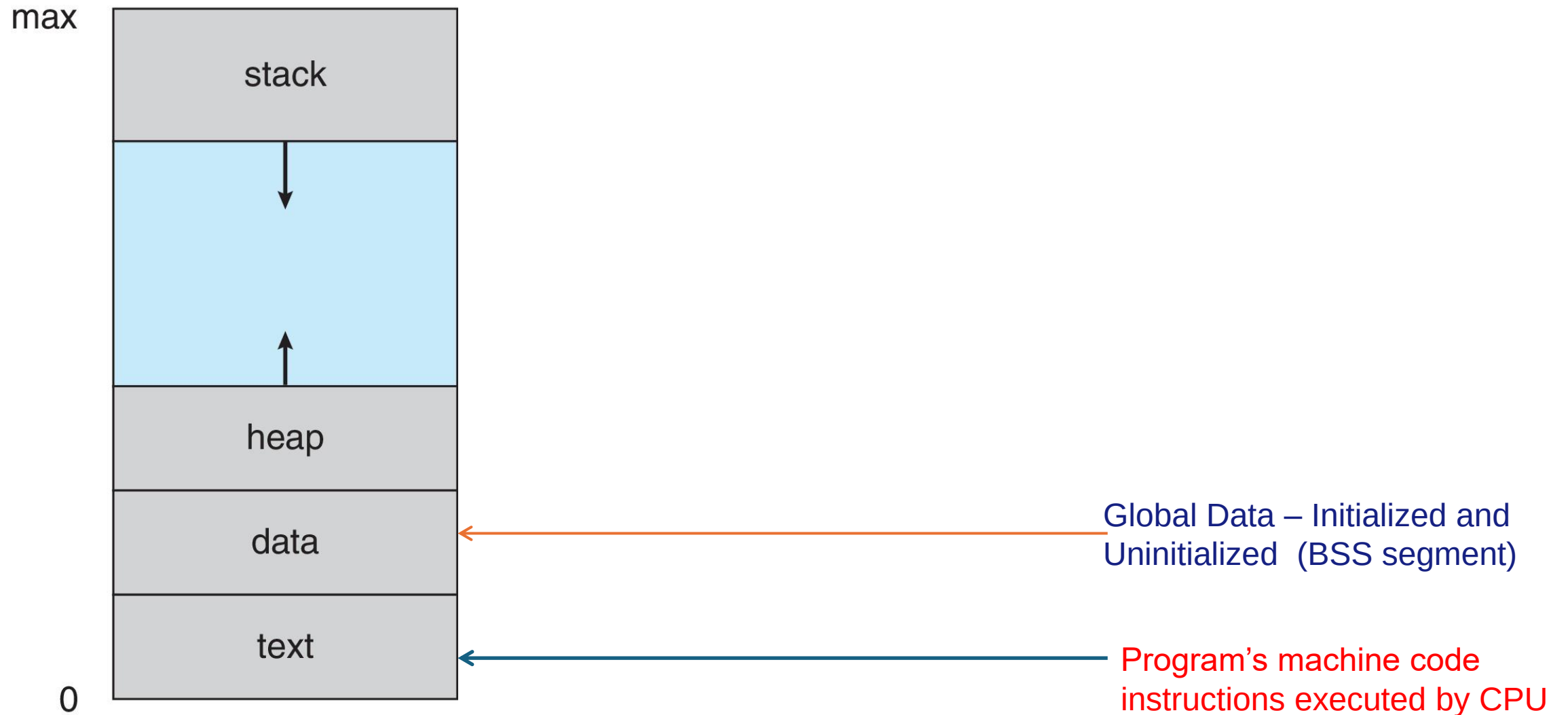
Process – Memory Layout

- A process in an operating system has its own virtual address space, separate from other processes. This virtual space is typically divided into distinct regions:
 - Text
 - Data
 - Heap
 - Stack

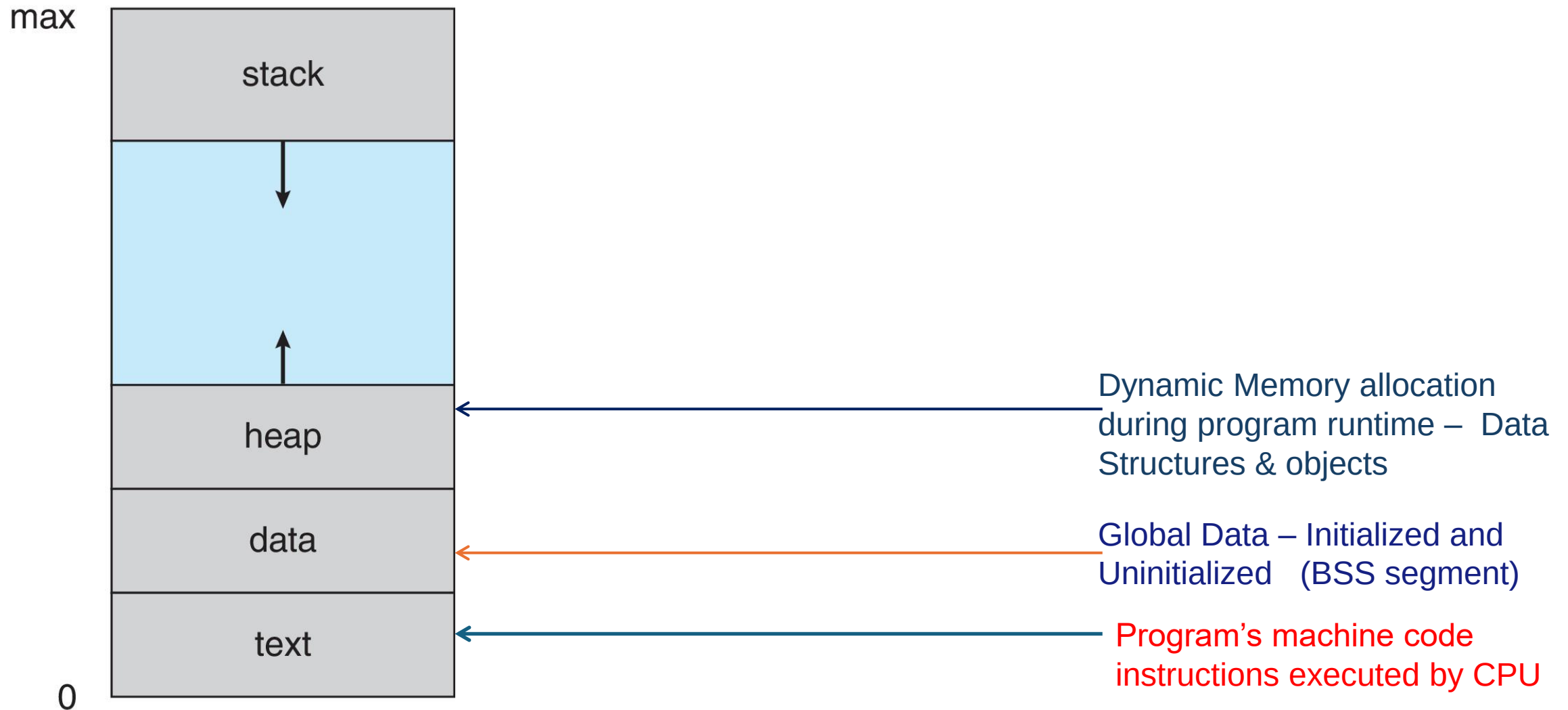
Process – Memory Layout



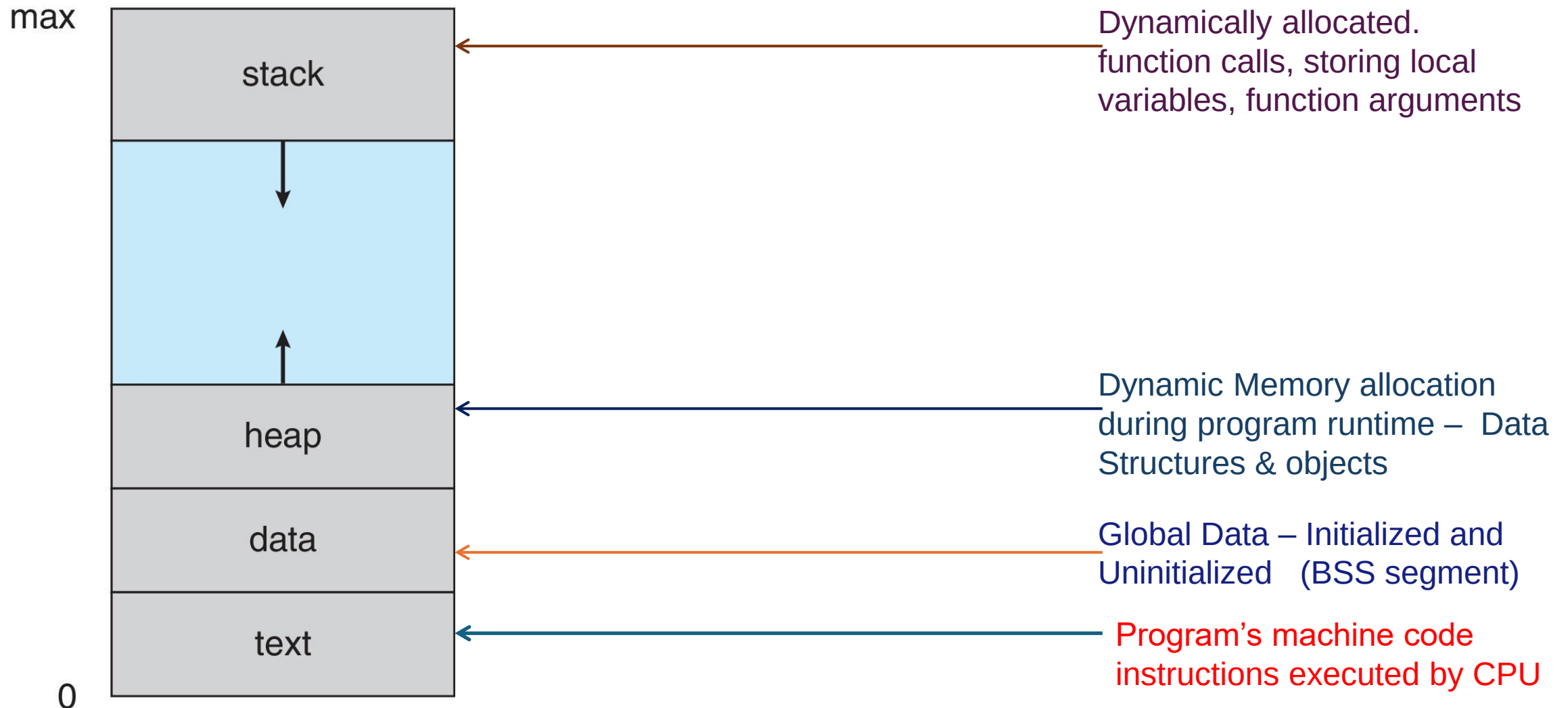
Process – Memory Layout



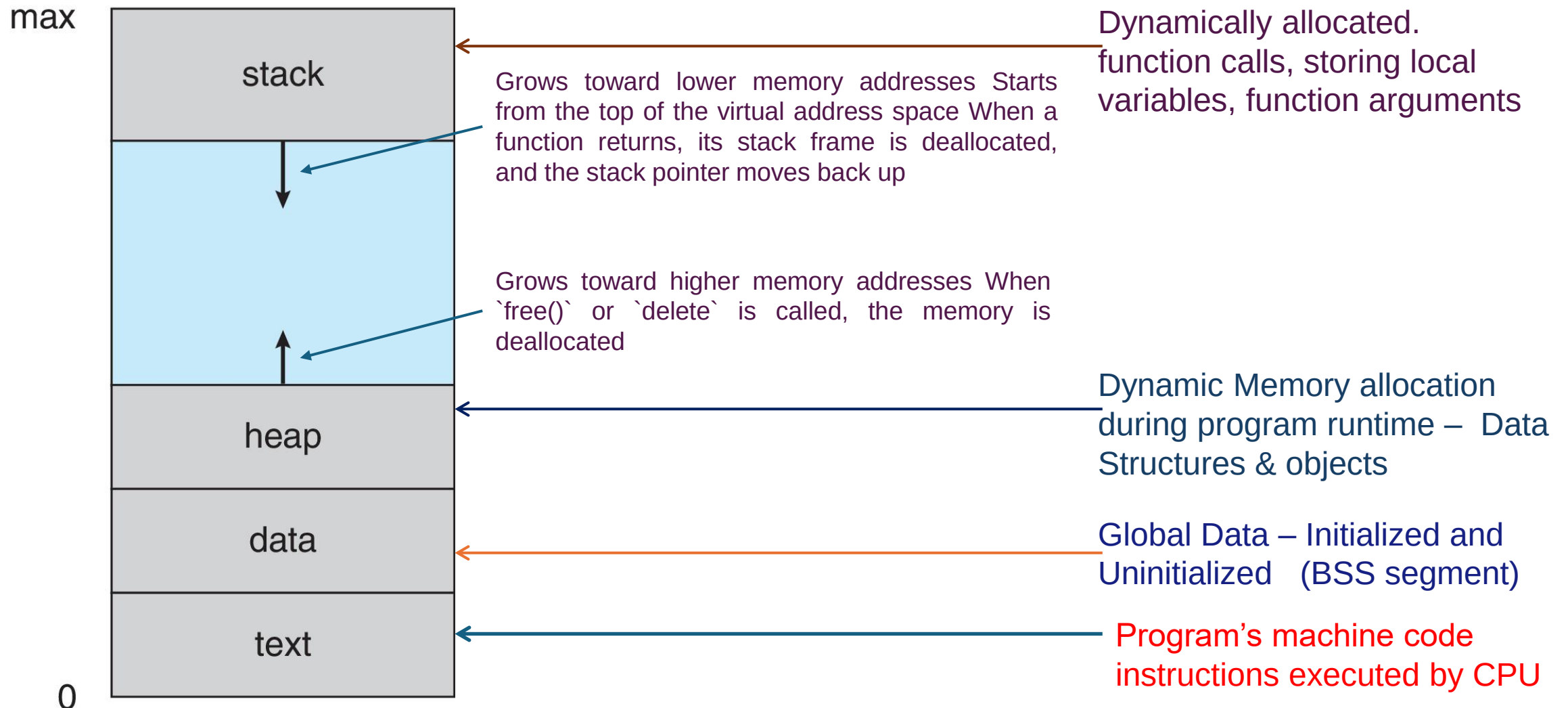
Process – Memory Layout



Process – Memory Layout



Process – Memory Layout



Process – Memory Layout - Example

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;    // Global variable

void my_function() {    // Function with local variables

    int local_var = 20;
    char *ptr = malloc(10);
    printf("Inside function:\n");
    printf("Global variable: %d\n", global_var);
    printf("Local variable: %d\n", local_var);
    printf("Malloced memory: %s\n", ptr);
    free(ptr);
}

int main() {
    my_function();
    printf("Outside function:\n");
    printf("Global variable: %d\n", global_var);
    return 0;
}
```

Process – Memory Layout - Example

```
#include <stdio.h>
#include <stdlib.h>
int global_var = 10;
void my_function() {
    int local_var = 20;
    char *ptr = malloc(10);
    printf("Inside function:\n");
    printf("Global variable: %d\n", global_var);
    printf("Local variable: %d\n", local_var);
    printf("Mallocated memory: %s\n", ptr);
    free(ptr);
}

int main() {
    my_function();
    printf("Outside function:\n");
    printf("Global variable: %d\n", global_var);
    return 0;
}
```

Program converted to Machine Code and this machine code is stored in **text section**

Process – Memory Layout - Example

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int global_var = 10;
```

```
void my_function() {
```

```
    int local_var = 20;
```

```
    char *ptr = malloc(10);
```

```
    printf("Inside function:\n");
```

```
    printf("Global variable: %d\n", global_var);
```

```
    printf("Local variable: %d\n", local_var);
```

```
    printf("Malloced memory: %s\n", ptr);
```

```
    free(ptr);
```

```
}
```

```
int main() {
```

```
    my_function();
```

```
    printf("Outside function:\n");
```

```
    printf("Global variable: %d\n", global_var);
```

```
    return 0;
```

```
}
```

Global variable which is initialized and will be stored in Data section and no uninitialized variable available so (block starting symbol) BSS section will be empty.

Process – Memory Layout - Example

```
#include <stdio.h>
#include <stdlib.h>
int global_var = 10;
void my_function() {
    int local_var = 20;
    char *ptr = malloc(10);
    printf("Inside function:\n");
    printf("Global variable: %d\n", global_var);
    printf("Local variable: %d\n", local_var);
    printf("Mallocated memory: %s\n", ptr);
    free(ptr);
}
int main() {
    my_function();
    printf("Outside function:\n");
    printf("Global variable: %d\n", global_var);
    return 0;
}
```

The diagram uses blue brackets and lines to group code blocks and link them to explanatory text. A bracket groups the function definition and its call in main(). Another bracket groups the local variable declaration and the malloc call. A third bracket groups the printf statements that output the values of global_var, local_var, and ptr.

My_function () and its local variables local_var will be stored in **Stack**.

In case of char *ptr variable only the variable and return address will be stored in the **stack**.
(local variables, Function parameters, return address)

Int main() will be stored in **Stack**.

Process – Memory Layout - Example

```
#include <stdio.h>
#include <stdlib.h>
int global_var = 10;
void my_function() {
    int local_var = 20;
    char *ptr = malloc(10);
    printf("Inside function:\n");
    printf("Global variable: %d\n", global_var);
    printf("Local variable: %d\n", local_var);
    printf("Malloced memory: %s\n", ptr);
    free(ptr);
}

int main() {
    my_function();
    printf("Outside function:\n");
    printf("Global variable: %d\n", global_var);
    return 0;
}
```

The program is asking for 10 bytes of memory while in executing status so heap segment will provide memory in runtime and all the intermediary data will be stored in heap segment

Process – Information Stored in

- The program code, also called text section

Current activity including program counter, processor registers

- Stack containing temporary data

Function parameters, return addresses, local variables

- Data section containing global variables
- Heap containing memory dynamically allocated during run time

Process – Memory Layout

- **Text Segment (Code):** This region stores the program's machine code instructions that the CPU executes. It's often read-only and can be shared among processes using the same program or libraries.
- **Data Segment:** This area holds the process's statically allocated data, like global variables defined in the program code.

Process – Memory Layout

- **Heap:** This is a dynamically allocated memory region. Processes can request memory from the heap during runtime to store data structures or objects they create. The heap grows and shrinks as needed.
- **Stack:** The stack is another dynamically allocated region that grows in the opposite direction of the heap. It's used for function calls, storing local variables, function arguments, and the return address for each function.

Process – Memory Layout

- Each time a function is called, an **activation record** containing function parameters, local variables, and the return address is pushed onto the stack; when control is returned from the function, the activation record is popped from the stack.
- Although the stack and heap sections grow **toward** one another, the operating system must ensure they do not **overlap** one another

Process – Memory Layout – Example 2

```
#include <stdio.h>
#include <stdlib.h>

int x;
int y = 15;

int main(int argc, char *argv[])
{
    int *values;
    int i;

    values = (int *)malloc(sizeof(int)*5);

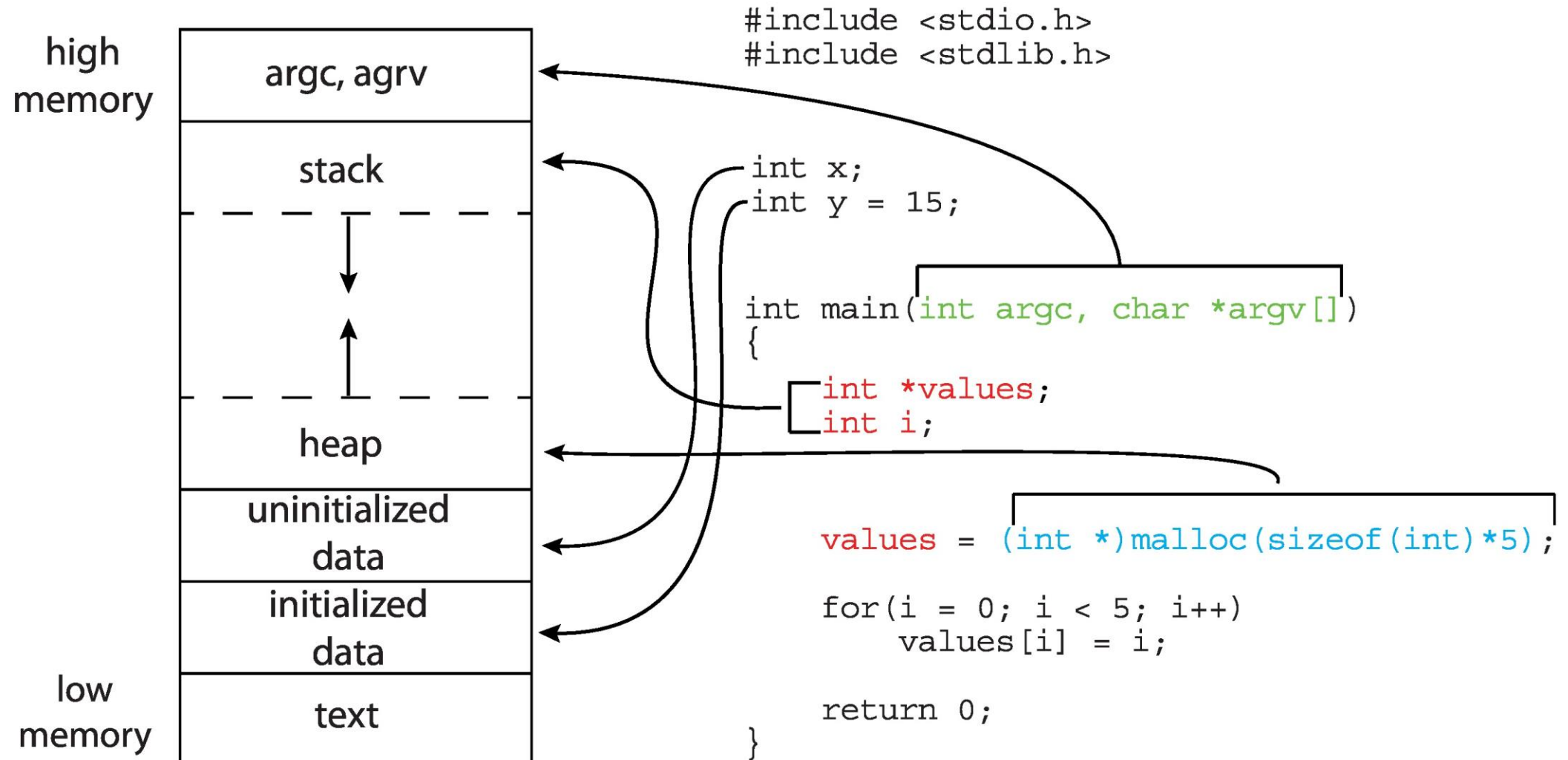
    for(i = 0; i < 5; i++)
        values[i] = i;

    return 0;
}
```

The diagram illustrates the memory layout of the provided C code. On the left, there are five horizontal arrows pointing to the right, representing memory segments. Arrows from the code point to these segments as follows:

- The first arrow (top) points to the header files: `#include <stdio.h>` and `#include <stdlib.h>`.
- The second arrow points to the global variables: `int x;` and `int y = 15;`.
- The third arrow points to the function parameters and local variables: `int main(int argc, char *argv[])`, `{`, `int *values;`, and `int i;`.
- The fourth arrow points to the memory allocation and the start of the loop: `values = (int *)malloc(sizeof(int)*5);` and `for(i = 0; i < 5; i++)`.
- The fifth arrow (bottom) points to the loop body and return statement: `values[i] = i;` and `return 0;`.

Process – Memory Layout – Example 2



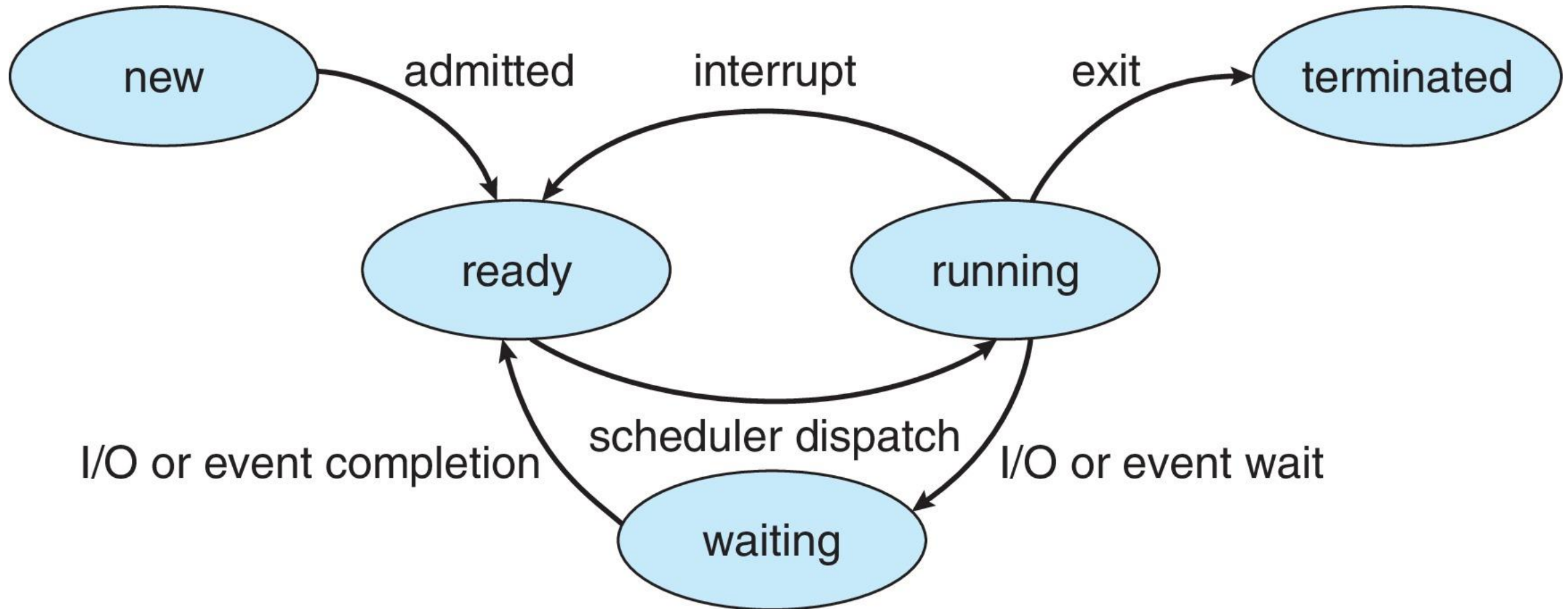
Process – Memory Layout

- It describes the organization of the memory for a process.
- The operating system's memory management unit (MMU) translates virtual addresses used by the process into physical memory addresses.

Process – States

- As a process executes, it changes state. The state of a process is defined in part by the current activity of that process.
- **A process may be in one of the following states:**
 - **New.** The process is being created.
 - **Running.** Instructions are being executed.
 - **Waiting.** The process is waiting for some event to occur (such as an I/O completion or reception of a signal).
 - **Ready.** The process is waiting to be assigned to a processor.
 - **Terminated.** The process has finished execution.

Process – States



Process – Process Control Block (PCB)

- A Data structure containing all the information about a process
- The PCB contains pointers to the process's memory layout.
- Each process is represented in the operating system by a **process control block** (PCB)—also called a *task control block*.
- The PCB acts as the handle for the OS to manage the process's memory layout, scheduling and other resources.

Process – Process Control Block (PCB)

Process ID
Process State
Process Priority
Program Counter
CPU registers
Heap & Stack Memory Limits
I/O Permissions & Status
CPU Scheduling
Accounting Information
List of Open files
.....
Pointers

Process – Process Control Block (PCB)

- **Process state.** The state may be new, ready, running, waiting, halted, and so on.
- **Program counter.** The counter indicates the address of the next instruction to be executed for that process.
- **CPU registers.** The registers vary in number and type, depending on the computer architecture. They include **accumulators, index registers, stack pointers, and general-purpose registers**, plus any condition-code information. Along with the program counter, this state information must be saved when an interrupt occurs, to allow the process to be continued correctly afterward when it is rescheduled to run.

Process – Process Control Block (PCB)

- **CPU-scheduling information.** This information includes a process priority, pointers to scheduling queues, and any other scheduling parameters.
- **Memory-management information.** This information may include such items as the value of the base and limit registers and the page tables, or the segment tables, depending on the memory system used by the operating system.
- **Accounting information.** This information includes the amount of CPU used, time limits, account numbers, job or process numbers, and so on.
- **I/O status information.** This information includes the list of I/O devices allocated to the process, a list of open files, and so on.

Process – Process Table

- A **data structure** maintained by the operating system to **store information about all active processes**.
- It's a table or array of process control blocks (PCBs).
- Each entry in the process table represents a **process** and contains a **pointer to its corresponding PCB**.

Process – Creation

- The act of creating a new process in an operating system.
- A process can create several new processes through creating process system calls during the process execution.
- The creating process is called a **parent process**, and the new processes are called the **children** of that process.
- Each of these new processes may in turn create other processes, forming a **tree of processes**.

Process – Creation

- Identify processes according to a **unique process identifier** (or pid), which is typically an integer number.
- The pid provides a unique value for each process in the system, and it can be used as an index to access various attributes of a process within the kernel.

Process – Creation

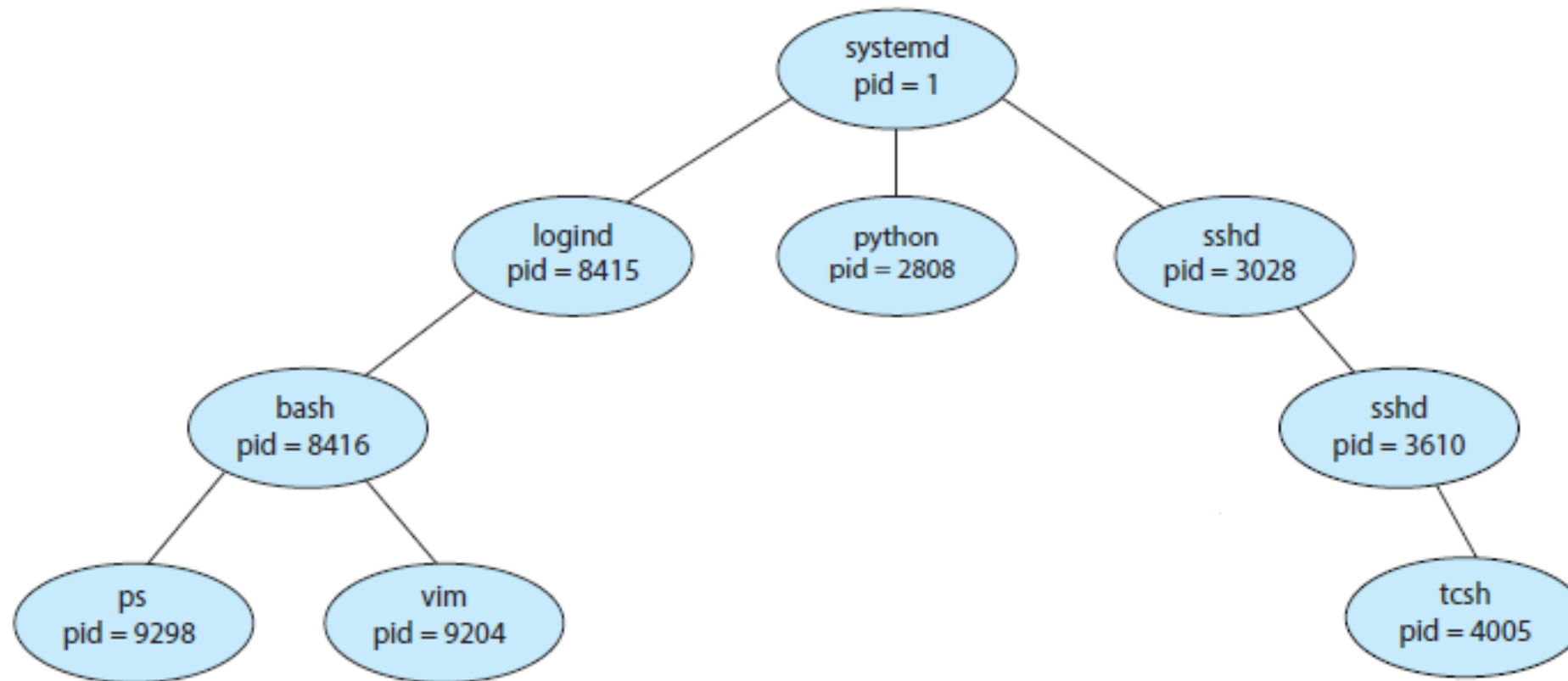


Figure: A tree of processes on a typical Linux system.

Process – Creation

- systemd (pid of 1) is the first daemon to start during booting and the last daemon to terminate during shutdown.
- It creates processes which provide additional services such as a web or print server, an ssh server, etc.
- The logind process is responsible for managing clients that directly log onto the system and the sshd process is responsible for managing clients that connect to the system by using ssh

Process – Parent Process and Child Process

- How Resource Allocation, Resource Sharing, Address Space and Execution happens when child processes are created?

Process – Parent Process and Child Process

Resource Allocation

- When a child process is created, it needs **resources** like **CPU time, memory, files, and I/O devices** to execute.
 1. The child process can **obtain resources directly from the OS**
 2. The child process can **inherit a subset of resources from the parent** process (The parent may have to partition its resources among its children)

Process – Parent Process and Child Process

Process Execution

- When a new process (child) is created, there are two possibilities for execution:
 1. **Concurrent Execution:** The parent process continues executing simultaneously with its child processes.
 2. **Sequential Execution:** The parent process waits until one or all of its child processes have finished executing before continuing.

Process – Parent Process and Child Process

Address Layout

When a new (child) process is created, there are two possibilities for its address space:

1. **Duplicate Address Space:** The child process has the same program and data as the parent process
2. **New Address Space:** The child process has a new program loaded into it, with its own separate address space.

Process – Parent Process and Child Process - Example

- Consider Unix System Calls `fork()`, `Wait()`, `exec()` and `exit()`
 - **fork()** - Creates a new process, called the child process, which is a copy of the current process, called the parent process. The child process starts executing from the point where `fork()` was called.
 - **wait()** - Suspends the current/parent process until one of its child processes terminates. The `wstatus` parameter is used to store the exit status of the terminated child process.

Process – Parent Process and Child Process - Example

- Consider Unix System Calls `fork()`, `Wait()`, `exec()` and `exit()`
 - **exec()** - Replaces the current/child process image with a new program loaded from the specified file.
 - **exit()** - Terminates the current process and returns the specified status code to the parent process. The parent process can use this status code to determine the reason for the child process's termination.

Creating a separate process using the UNIX fork() system call.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }

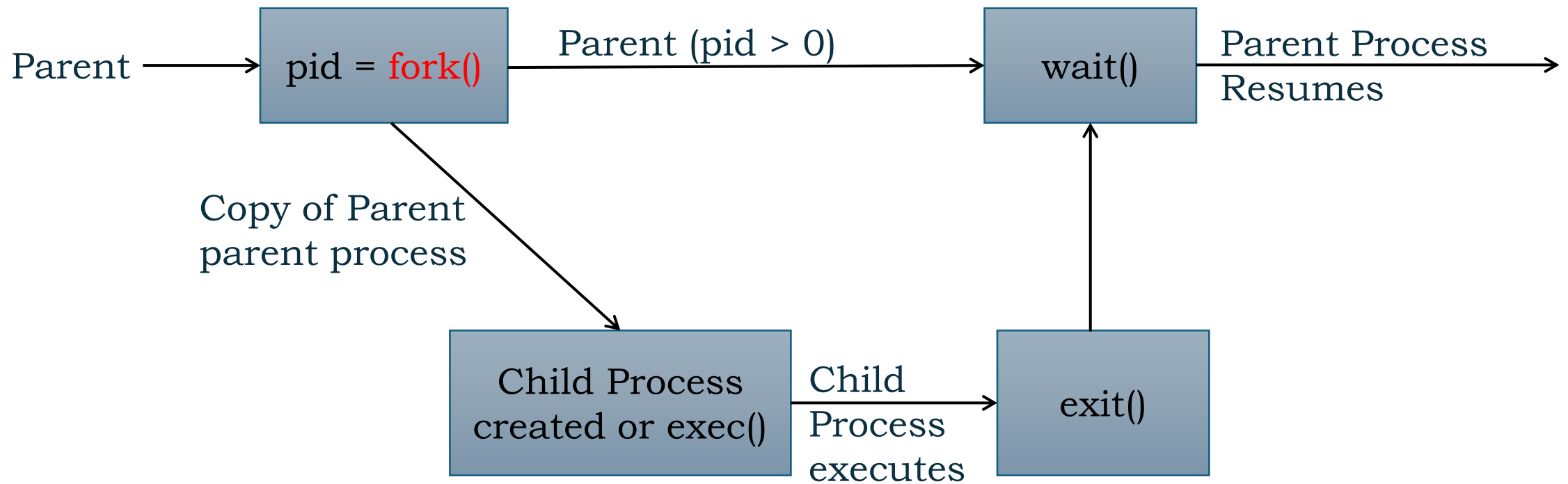
    return 0;
}
```

Process – Parent Process and Child Process - Example

- Consider Unix System Calls `fork()`, `wait()`, `exec()` and `exit()`

Time →

Parent-Child Process Execution



Process Termination

- A process terminates when it finishes executing its final statement and asks the operating system to delete it by using the **exit()** system call.
- All the resources of the process are deallocated and reclaimed by the operating system.
- A parent process can terminate another process using **TerminateProcess()**.
- A user or a misbehaving application could arbitrarily kill another user's processes.

Process Termination

- A parent may terminate the execution of one of its children for a variety of reasons:
 - The child has exceeded its usage of the allocated resources.
 - The task assigned to the child is no longer required.
 - The parent is exiting
- **Cascading Termination** - if a parent process terminates (either normally or abnormally), then all its children must also be terminated.

Process Termination

- A parent process may wait for the termination of a child process by using the `wait()` system call.
- The `wait()` system call is passed a parameter that allows the parent to obtain the exit status of the child.

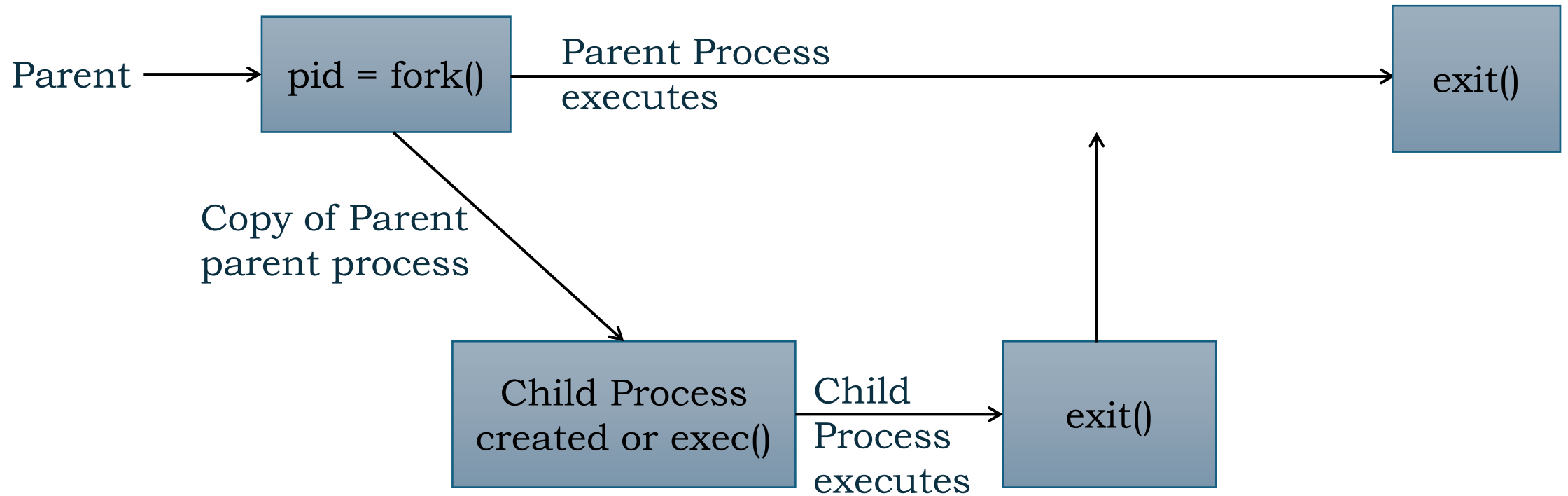
`pid = wait(&status);`

- When a process terminates, its resources are deallocated by the operating system. However, its **entry in the process table must remain** there until the parent calls `wait()`

Process – Parent Process and Child Process - Example

Circumstance 1 – Zombie Process

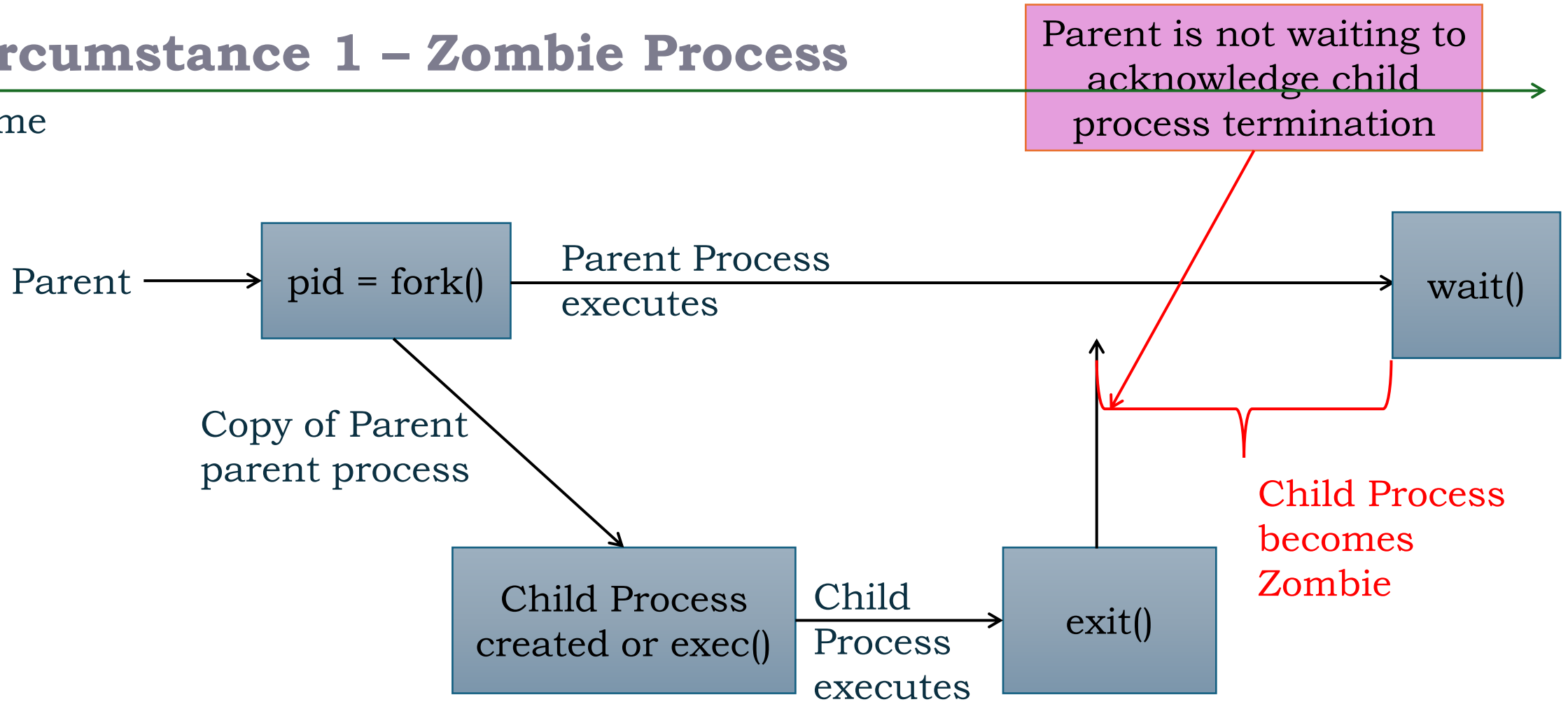
Time →



Process – Parent Process and Child Process - Example

Circumstance 1 – Zombie Process

Time



Process – Parent Process and Child Process - Example

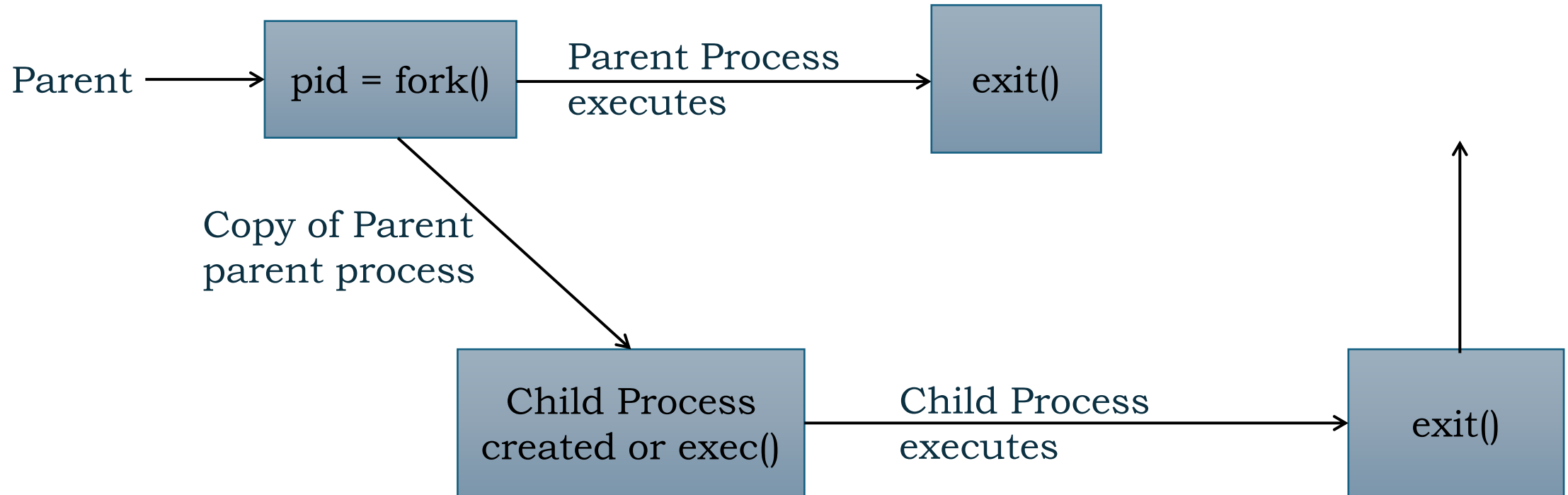
Circumstance 1 – Zombie Process

- A zombie process is a process that has finished executing, but its parent process has not yet acknowledged its termination.
- The zombie process still exists in the process table, but it is no longer running.
- The parent process is responsible for waiting for the zombie process to finish and retrieving its exit status using the ``wait()`` system call.
- If the parent process does not wait for the zombie process, it will remain in the process table until the parent process terminates or the system reboots.

Process – Parent Process and Child Process - Example

Circumstance 2 – Orphan Process

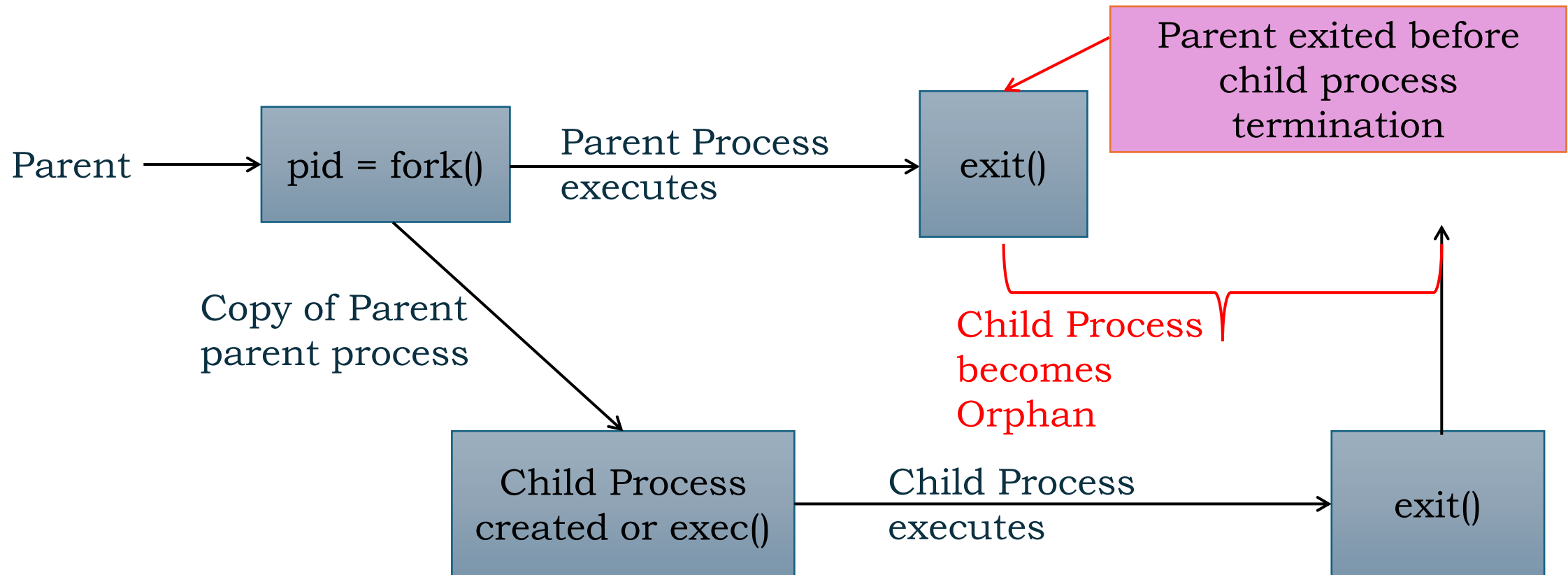
Time →



Process – Parent Process and Child Process - Example

Circumstance 2 – Orphan Process

Time →



Process – Parent Process and Child Process - Example

Circumstance 2 – Orphan Process

- An orphan process is a process whose parent process has terminated, but the child process is still running.
- The orphan process is adopted by the `init` process (PID 1), which becomes its new parent.
- The `UNIX` systems addressed this scenario by assigning the **init process** as the new parent to orphan processes.

Process – Parent Process and Child Process - Example

Circumstance 2 – Orphan Process

- The **init process** periodically invokes `wait()`, thereby allowing the exit status of any orphaned process to be collected and releasing the orphan's process identifier and process-table entry.

Android Process Hierarchy

- Because of resource constraints such as limited memory, mobile operating systems may have to terminate existing processes.
- Rather than terminating an arbitrary process, Android has identified an ***importance hierarchy*** of processes, and therefore it terminates processes in order of increasing importance.

Android Process Hierarchy

- **Foreground process**—The current process visible on the screen
- **Visible process**— A process performing an activity whose status is displayed on the foreground process.
- **Service process**—A process that is performing an activity that is apparent to the user.
- **Background process**—A process that may be performing an activity but is not apparent to the user.
- **Empty process**—A process that holds no active components associated with any application